SOP: Rightsizing Explainer, Step-by-Step Manual

Oct 30, 2026 · @Ved

1. Purpose and outcome

Follow this manual end to end and you will run a tool that tells you whether a Kubernetes CPU or memory recommendation is safe to apply, and explains why in plain English. It takes about two to three hours of hands-on time, plus waiting for metrics to build up.

Who this is for: an engineer new to Kubernetes operations and observability. You should be comfortable with a Linux terminal and basic `kubectl`. You do not need to know Prometheus, KRR or LLMs beforehand; each is explained where it first appears.

By the end you will have:

- A local Kubernetes cluster running five test workloads, each with a known problem.
- Evidence files built from real Prometheus metrics, one per workload container.
- A verdict per workload, such as "safe to lower" or "do NOT lower: it is being throttled".
- A plain-language explanation of each verdict written by a local LLM, with every number in it checked by code.
- A benchmark showing how reliably the model follows the rules.

How to use this manual: do the parts in order. Each part has the command, what you should see, and **Why we do this**. If your output differs from the expected output, go to the Troubleshooting section before continuing.

2. Concepts you must know first

Every decision in this project rests on twelve ideas. Read this table once now and come back to it whenever a term is unclear.

Term	What it means	Why it matters here
Request	The amount of CPU or memory Kubernetes reserves for a container when scheduling it.	This is the number we are trying to get right. Too high wastes money; too low risks instability.

Term	What it means	Why it matters here
Limit	The maximum a container may use. Over the CPU limit it is slowed down; over the memory limit it is killed.	Limits turn a sizing mistake into a visible failure.
CPU throttling	The Linux scheduler pauses a container that has used its CPU quota in a 100 ms period.	A throttled container looks idle in usage graphs but is actually starved. Cutting its request makes things worse.
OOMKill	The kernel kills a container that exceeds its memory limit.	Proof that real memory demand was higher than what was measured.
Working set	The memory the kernel considers actively in use and not easily reclaimed.	The memory number that best predicts an OOMKill, so we size from it.
p95	The value that 95% of samples fall below.	A standard way to size CPU while ignoring rare spikes. Rare spikes can still hurt, so we also track a 1-minute peak.
Prometheus	A metrics database that scrapes your cluster every ~30 seconds.	Our single source of usage data.
KRR	Kubernetes Resource Recommender, an open-source tool by Robusta that proposes requests from Prometheus data.	We use its real output as one input. We do not rebuild it.
Evidence pack	One JSON file per workload container holding every number the project may quote. Missing data is `null`, never `0`.	The single source of truth. Everything downstream reads only this.
Verdict	The rules engine's decision for CPU and memory, with the reasons behind it.	The decision is made by code, not by the LLM.
Policy profile	A set of thresholds (`lab`, `dev`, `production`) such as how much history is required.	Different environments tolerate different risk.
Ollama	A program that runs LLMs locally on your own GPU or CPU.	The explainer uses it, so no data leaves your machine.

3. The problem and the one design rule

A rightsizing tool gives you a number, but a number alone is not safe to apply. Lowering a request because usage looks low can make an application slower or crash it, and the tool rarely tells you which cases are risky.

Three real examples from the test cluster show why:

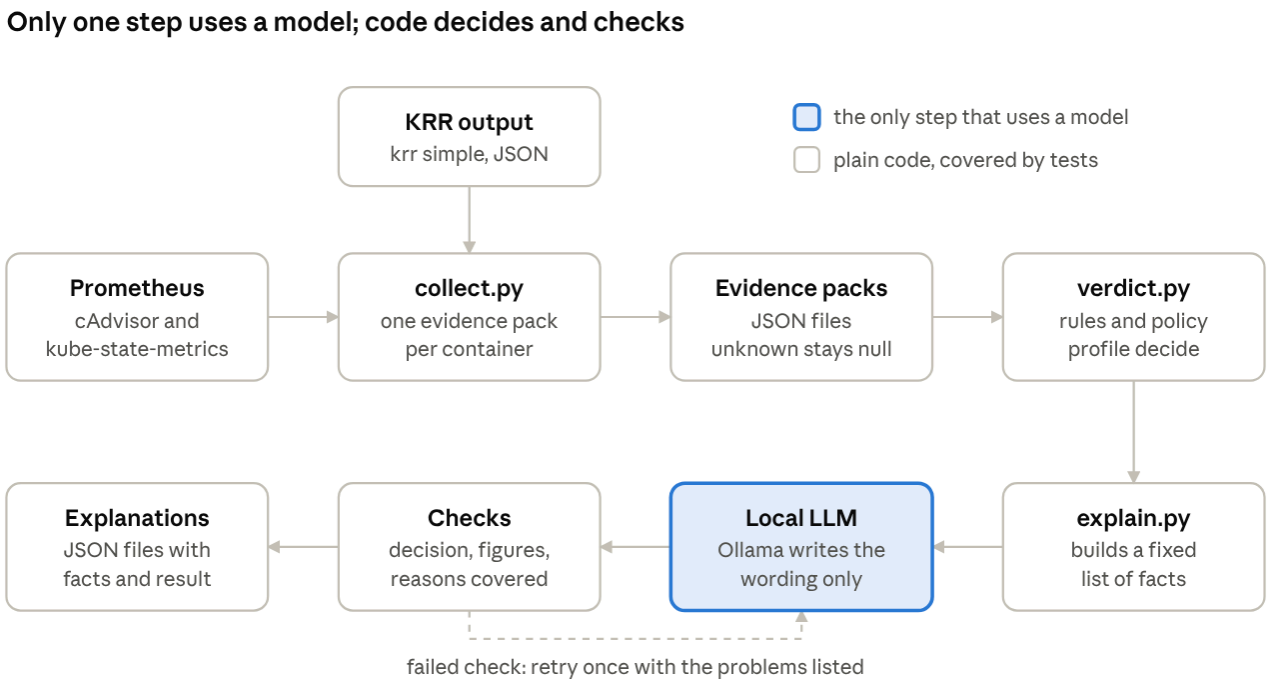
- A container used only 0.037 cores against a 0.1-core limit, so it looked over-provisioned. In fact it was throttled in 60% of CPU periods, and its true demand was higher than what was measured.
- A memory-leaking container showed a 36 MiB peak after a restart, but its real peak before the restart was about 124 MiB, close to its 128 MiB limit.
- KRR recommended 0.409 cores for a bursty workload where a simple 5-minute p95 said 0.108. The sampling resolution changed the answer by almost four times.

The one design rule: code decides, the model explains. Every number and every decision comes from deterministic code that you can test. The LLM only turns the result into readable sentences, and its output is rejected by code if it breaks the rules.

Why we do this: LLMs are good at wording and unreliable at arithmetic and facts. If the model decided safety, one confident wrong sentence could cause an outage. Splitting the job keeps the safety logic testable and uses the model only where it is strong.

4. How the system works

Data flows one way, from Prometheus through two code stages to the explainer, and only the explainer's wording step uses a model.



data flow from Prometheus to a checked explanation · 8 stages, 1 uses a model

Read the picture in two lines. The top row turns metrics into evidence packs and then into verdicts, all in plain code. The bottom row, read right to left, builds a fixed list of facts, lets the local LLM word it, and has code check the result. A failed check sends the model one more attempt, with its mistakes listed.

Stage	Command	Reads	Writes
Collect	`make collect`	Prometheus, plus the KRR JSON if you pass `KRR=`	`evidence/<namespace>/<workload>__<container>.json`
Verdict	`make verdict`	The evidence packs	`verdicts/<namespace>/...json`
Explain	`make explain`	Evidence packs and verdicts	`explanations/<namespace>/...json`
Benchmark	`make benchmark`	12 synthetic cases plus your evidence packs	`benchmark/results.md` and `results.json`

Why we do this: each stage writes files and the next stage reads only those files. That makes every step repeatable, lets you inspect what each stage produced, and means the tests can run without a cluster.

5. Before you start: tools checklist

Run each command. Every row must pass before Part A. A missing tool is the most common reason a first run fails.

Tool	Check command	Expected	Why we need it
Docker	`docker ps`	A table (may be empty), no "cannot connect" error	kind runs the cluster inside Docker containers
kind	`kind version`	A version number	Creates the local Kubernetes cluster
kubectl	`kubectl version --client`	A client version	Talks to the cluster
make	`make --version`	GNU Make version	Every project step is a `make` target
Python 3.11+	`python3 --version`	3.11 or newer	The project itself uses only the standard library, so no `pip install` is needed
Python 3.10 to 3.12 (separate)	`python --version` inside the `krr` environment	3.12.x	KRR refuses to install on newer Python (see Part E)
Ollama	`ollama ls`	At least one model listed	Runs the local LLM
A model	`ollama ls` shows e.g. `qwen2.5-coder:14b`	Model name and size	Writes the explanations. About 9 GB for a 14B model
GPU (optional)	`nvidia-smi`	A GPU table	A 14B model answers in about 6 seconds on a 16 GB GPU; on CPU it is far slower

You also need two Git repositories on disk:

- The **resource-review mixin** repo, which provides the kind cluster with Prometheus and Grafana (`make dev` in that repo).
- This **rightsize-explainer** project folder (the folder with `Makefile` , `src/` , `tests/` and `scenarios/`).

Why we do this: most failures in the first hour are a stopped Docker daemon or a missing tool, not a code problem. Checking first turns a confusing error later into a clear one now.

6. Part A: start the environment

Goal: a running kind cluster with Prometheus answering on port 9090.

1. Start Docker if it is not running:

   ```
   sudo systemctl start docker
   docker ps
   ```

   Expected: a table, possibly with the container `resource-review-mixin-control-plane`. An error about `/var/run/docker.sock` means Docker is still stopped.

2. If the cluster container exists but is stopped, start it:

   ```
   docker start resource-review-mixin-control-plane
   kubectl get pods -A
   ```

   Expected: all pods `Running` within one to two minutes.

3. If the cluster does not exist, go to the **mixin repo folder** (not this project's folder) and run:

   ```
   make dev
   ```

   This creates the kind cluster and installs Prometheus and Grafana. It takes several minutes the first time.

4. Confirm Prometheus answers:

   ```
   curl -s 'http://127.0.0.1:9090/api/v1/query?query=up' | head -c 200
   ```

   Expected: JSON that starts with `{"status":"success"`. If you get nothing, Prometheus is not reachable yet. Wait a minute and retry.

Why we do this: every later step reads from Prometheus. If it is not answering now, `make collect` will fail with "cannot reach Prometheus" and you will not know whether the cause is the cluster or the code.

Common mistake: running `make dev` inside the rightsize-explainer folder. That folder has no `dev` target and make prints `No rule to make target 'dev'`. The command belongs in the mixin repo.

7. Part B: deploy the lab workloads

Goal: five workloads in the `explainer-lab` namespace, each built to show one rightsizing situation.

1. From the rightsize-explainer folder:

   ```
   cd /media/ved/DATA/K8/capstone/rightsize-explainer
   make lab
   kubectl -n explainer-lab get pods
   ```

   Expected: pods for `steady-overprovisioned` (2), `throttled-cpu`, `leaky-memory` and `bursty-rare` reach `Running`. `batch-periodic` is a CronJob that starts a short pod every 20 minutes, so its pod appears later.
2. **Wait for history.** Prometheus only knows what happened after it started scraping these pods.

What you want to do	Minimum wait
See any numbers in `make collect`	about 10 minutes
Get verdicts under the `lab` profile	20 minutes of history
Run KRR (it needs 100 data points)	about 3 to 4 hours
See `batch-periodic` properly (several 20-minute runs)	about 1 hour

The five workloads, and what each teaches:

Workload	Behaviour	Situation it represents
steady-overprovisioned	Idle nginx, 2 replicas, request 500m CPU and 512 MiB	Money wasted on resources never used
throttled-cpu	Wants about 0.3 cores, limit 0.1 cores	Looks idle but is starved, so a cut would hurt
leaky-memory	Grows 8 MB per minute, limit 128 MiB	Gets OOMKilled and restarts repeatedly
bursty-rare	30-second CPU burst every 5 minutes	Percentiles hide short, intense peaks
batch-periodic	CronJob, burns CPU for up to 3 minutes every 20 minutes	Short-lived pods; naive per-pod history is misleading

Why we do this: a real cluster rarely gives you a clean example of each failure mode on demand. Building them on purpose lets you check that the tool reaches the right conclusion for each one, and every later result in this manual can be compared with a known cause.

Common mistake: collecting after only a few minutes. Numbers will be empty or tiny and the verdicts will say `insufficient_evidence`. That is the tool working correctly, not a bug.

8. Part C: run the tests

Goal: confirm the code works on your machine before trusting any output.

```
make test
```

Expected: the last lines read `Ran 81 tests` and `OK`. Lines beginning with `error:` in the middle are normal. They come from tests that deliberately trigger an error to check the message.

The tests need no cluster and no GPU. They use a fake Prometheus and a fake Ollama that run inside the test process.

Why we do this: a passing suite tells you the code on your disk is complete and consistent. A wrong test count means a file was not copied into place. For example, 32 tests instead of 45 meant new files had not been copied yet.

If tests fail: read the first `ERROR` or `FAIL` block. `ModuleNotFoundError` or `has no attribute` almost always means a file is missing from `src/explainer/` or is an old version.

9. Part D: collect evidence packs

Goal: turn raw Prometheus metrics into one JSON evidence pack per workload container.

```
make collect WINDOW=6h
```

`WINDOW` is the maximum history to read. If Prometheus holds less, the tool uses what exists and records that in the pack. The command prints a table:

Column	Meaning
hist(h)	Hours of history actually found
cpu p95	95th-percentile CPU use in cores

Column	Meaning
cpu req	The configured CPU request in cores
pk 1m	The highest 1-minute CPU rate, which catches short bursts
mem max	Peak working-set memory
mem req	The configured memory request
throttle	Share of CPU periods in which the container was throttled (`-` means no CPU limit, so unknown)
restarts	Container restarts in the window
oom	Whether an OOM kill was seen

Files are written to `evidence/explainer-lab/<workload>__<container>.json`. Open one and read it. It contains the workload identity, the configured requests and limits, the observed figures, and a `limitations` list that states what the data cannot tell you.

Why we do this: collecting once into files, instead of querying live every time, makes every later step repeatable and testable. The LLM never sees Prometheus. It only ever sees what is in these files.

Four design choices you should understand:

1. **Missing data is `null`, never `0`.** Zero CPU and unknown CPU mean different things, and mixing them leads to wrong cuts.
2. **Pods of one workload are merged using the highest value.** This is conservative, so it avoids under-sizing.
3. **Restarted containers are merged.** A restart creates a second metric series with the same name. Without merging, `leaky-memory` showed 36 MiB when its real peak was about 124 MiB.
4. **History is measured per workload, not per pod.** CronJob pods live about 4 minutes. Per-pod counting reported 0.1 hours of history for a workload that had run for over 4 hours.

Common mistake: the `hist(h)` column is much smaller than `WINDOW`. This is normal after a restart of the cluster, because Prometheus may have lost its older data. Wait and collect again.

10. Part E: get real KRR output

Goal: add KRR's own recommendation to each evidence pack, so the verdicts use a real tool's number and can be cross-checked.

Do this only after about 3 to 4 hours of history. KRR needs at least 100 data points per workload.

1. Install KRR in its own Python environment. KRR only supports Python 3.10 to 3.12. If your default Python is newer, pip silently skips every package ("Ignoring ... markers don't match") and KRR fails with `No module named 'click'`.

```
mkdir -p ~/tools && cd ~/tools
git clone https://github.com/robusta-dev/krr
cd krr
conda create -n krr python=3.12 -y
conda activate krr
python --version        # must say 3.12.x
pip install -r requirements.txt
python krr.py --help
```

2. Run KRR against your Prometheus and save the JSON:

```
python krr.py simple -p http://127.0.0.1:9090 -n explainer-lab \
  --history_duration 4 -f json --fileoutput ~/krr-lab.json --logtostderr
```

   `--history_duration 4` means four hours. KRR's default is 336 hours (14 days), far more than the lab holds. Match it to your real history, never more.

3. Collect again, now with KRR's file:

```
cd /media/ved/DATA/K8/capstone/rightsize-explainer
make collect WINDOW=6h KRR=~/krr-lab.json
```

What you will notice, and why it matters:

- KRR recommends **100 MiB of memory and 10m CPU for many small workloads**. These are KRR's own minimums (`--mem-min 100`, `--cpu-min 10`), not measured needs. The project detects a value equal to KRR's minimum and falls back to a usage-based figure, noting why (`I_KRR_AT_FLOOR`).
- KRR returns `?` with "Not enough data" for the CronJob, because it looks at each short-lived pod separately. The project then uses its own per-workload figure (`I_KRR_NO_DATA`).
- For `bursty-rare`, KRR's CPU figure (0.409 cores) is far above this project's 5-minute p95 (0.108 cores). The verdict flags this as `K_DISAGREES` so a human checks the peak before acting.

Why we do this: using a real tool's output makes the project more credible than re-implementing its formula, and comparing the two exposes weaknesses in both.

Note: `~/krr-lab.json` is your real KRR output in your home folder. `tests/fixtures/krr-lab.json` is a small copy used by the tests. They share a name but are different files. Never copy one over the other.

11. Part F: read the verdicts

Goal: apply the safety rules and understand each decision.

```
make verdict PROFILE=lab
```

Files go to `verdicts/explainer-lab/`. The table has one row per workload with a CPU verdict, a memory verdict and the reason codes behind them.

The seven possible verdicts:

Verdict	Meaning	Action
decrease_ok	Lowering the request is supported by the evidence	Safe to consider lowering
decrease_blocked	A lower candidate exists, but a safety rule blocks it	Do not lower; read the reasons
increase_supported	Evidence shows pressure a higher request would relieve	Consider raising
increase_optional	Candidate is higher but nothing shows suffering	Low priority
no_change	The difference is too small to matter	Leave as is
no_request_set	No request is configured, so there is nothing to compare	Set a request first
insufficient_evidence	Not enough history or data to decide	Wait and collect more

The reason codes you will meet most:

Code	Meaning	Effect
C_THROTTLED	Throttled in 5% or more of CPU periods	Blocks a CPU decrease

Code	Meaning	Effect
C_PEAK_ABOVE_CANDIDATE	1-minute peak is over 2x the candidate	Blocks a decrease; warns on an increase
M_OOM_SEEN	An OOM kill happened in the window	Blocks a memory decrease
M_NEAR_LIMIT	Peak memory is 90% or more of the limit	Blocks a memory decrease
M_RESTARTS	Container restarted in the window	Blocks a memory decrease
S_THROTTLED, S_OOM, S_NEAR_LIMIT	Same facts, read as support for an increase	Supports raising
S_USAGE_ABOVE_REQUEST	Usage is above the current request	Supports raising
E_SHORT_HISTORY	Less history than the profile requires	Prevents a confident decision
E_NO_DATA	No usable data	`insufficient_evidence`
K_DISAGREES	KRR and this tool differ by more than 25%	Warning: check the peak
I_FLOOR, I_IMMATERIAL, I_KRR_AT_FLOOR, I_KRR_NO_DATA	Informational notes	Explain how a number was adjusted

Open one verdict JSON. Every reason lists the **evidence fields** it used, with their values. A test checks that each cited field exists in the evidence pack with the same value, so a reason can never refer to a number that is not there.

Policy profiles:

Profile	History required	Use for
lab	20 minutes	This exercise only
dev	7 days	Development clusters
production	28 days	Real workloads

Try `make verdict PROFILE=production` and notice that every `decrease_ok` becomes `insufficient_evidence`, because the lab has hours of data and production demands weeks. Blocked decreases stay blocked, and increases stay supported but carry a short-history warning.

Why we do this: the decision is made by plain code with explicit thresholds, so you can test it, change it and defend it. The thresholds are illustrative values taken from examples in the LearnKube rightsizing book. They are not industry standards, so tune them to your own risk tolerance.

12. Part G: generate and check explanations

Goal: have the local model explain each verdict in plain language, with code checking every claim.

```
make explain ONLY=bursty      # one workload first
make explain                  # all workloads
```

The first call is slower because Ollama loads the model onto the GPU. After that, expect about 5 to 6 seconds per workload on a 16 GB GPU. Use `MODEL=name` to pick another installed model. The default is `qwen2.5-coder:14b`.

What happens inside each call:

1. The code builds a short **FACTS list** from the evidence and verdict. Each figure is already rounded and formatted the way it should be quoted, such as `0.409 cores` or `64Mi`.
2. The model receives the facts and strict rules: copy figures exactly, never calculate or convert units, mention every blocking, warning or supporting reason, and repeat the verdict it was given.
3. Ollama is called with temperature 0, a fixed seed and a JSON schema, so the reply is repeatable and always has the right shape.
4. Code checks the reply (see below). If a check fails, the model is shown exactly what it got wrong and gets **one** more attempt.
5. The result, including the facts, raw reply and whether it needed a retry, is saved to `explanations/explainer-lab/`.

The three checks:

Check	Fails when	Example
Decision	The model states a different decision from the verdict	Verdict says `decrease_blocked`, model says `decrease_ok`

Check	Fails when	Example
Figures	A number or unit appears that is not in the facts	The facts say `0.050 cores`, the model writes `0.5 cores` or `50m`
Reason coverage	A blocking, warning or supporting reason is not mentioned	A throttled workload's explanation never mentions throttling

A reason counts as covered if the text names its topic, or quotes every figure from that reason's fact. This second route exists because the first version wrongly rejected a correct sentence, "request 0.200 cores is below usage 0.613 cores", for not containing the word "above".

Why we do this: a fluent but wrong explanation is worse than none, because people trust fluent text. Checking the output with plain code is what lets you rely on a model that can, in principle, make things up.

What the checks do not cover: they verify decisions, figures and topic coverage. They cannot prove that every sentence is true or helpful. Read a few explanations yourself, and never apply a change just because the text sounds convincing.

The caveats shown under each explanation are chosen by code, not by the model, so only the relevant ones appear (limited history, intermittent workload, no KRR entry).

13. Part H: benchmark the model

Goal: measure how reliably a model follows the rules across many cases, instead of judging it from one good-looking example.

```
make benchmark MODELS=qwen2.5-coder:14b
```

The benchmark runs the model on 17 cases: 12 synthetic ones (hand-built to cover every verdict type, including no data, no request set, near the memory limit, restarts only, and the production profile) plus your 5 real lab workloads. It writes `benchmark/results.md` and `benchmark/results.json`.

Column	Meaning
Passed first try	Cases where decision, figures and reason coverage were all right on the first reply
Passed after retry	Cases that passed once the model saw its own mistakes
Avg seconds	Time per case

Our reference result on this project: 17 of 17 cases passed on the first try in about 6 seconds each. An earlier run scored 15 of 17, because the reason-coverage check was too strict. Fixing a wrong check, not the model, is a normal part of this kind of work.

To compare models, list more than one, separated by commas. Models must already be installed, otherwise the command prints the installed list and stops. Skip comparison if you only want a baseline.

Why we do this: one good answer proves little. A benchmark on a fixed set of cases tells you how often the model fails, which checks catch it, and whether a change you make improves or damages the result.

Read results honestly: a score of 100% on an easy test shows that the model passes that test, not that it is always right. The synthetic cases are invented. Add harder cases when you extend the project.

14. Shutting down and resuming

Your results are files on disk, so a shutdown loses nothing except Prometheus history. Kubernetes and Prometheus run in Docker, and Prometheus has lost its older data after a restart before.

To shut down:

1. Stop any terminal running a port-forward with Ctrl+C.
2. Optionally save a snapshot of today's results:

```
cd /media/ved/DATA/K8/capstone/rightsize-explainer
mkdir -p snapshots/$(date +%F)
cp -r evidence verdicts explanations benchmark ~/krr-lab.json
snapshots/$(date +%F)/
```

3. Stop the cluster cleanly so Docker does not have to recover it:

```
docker stop resource-review-mixin-control-plane
```

4. Power off.

To resume:

1. `sudo systemctl start docker`, then `docker start resource-review-mixin-control-plane`.
2. `kubectl get pods -A` and wait until everything is `Running`.
3. Check Prometheus: `curl -s 'http://127.0.0.1:9090/api/v1/query?query=up' | head -c 200`. If it does not answer, run `make dev` in the **mixin repo** folder.
4. Check the lab: `kubectl -n explainer-lab get pods`. If the namespace is empty, run `make lab`.
5. Run `make test` and expect 81 passing.
6. Wait one to two hours before `make collect WINDOW=1h`, because history may have reset.

Why we do this: a clean stop avoids corrupted state, and the snapshot gives you a record of results even if the cluster has to be rebuilt. Ollama usually starts by itself. If `make explain` cannot reach it, run `ollama serve`.

15. Troubleshooting

Every row below happened during the original build, so expect to meet them.

Symptom	Cause	Fix
`Cannot connect to the Docker daemon` or errors about `/var/run/docker.sock`	Docker is not running	`sudo systemctl start docker`
`make: *** No rule to make target 'dev'`	You ran `make dev` in the wrong folder	Run it in the mixin repo; this project has no `dev` target
`make: *** No rule to make target 'verdict'`	Your `Makefile` is an old copy	Copy the latest `Makefile` into the project root
`make test` runs fewer tests than expected (32 instead of 45 or 81)	New source or test files were not copied in	Copy every delivered file into `src/explainer/`, `tests/` and `tests/fixtures/`
`AttributeError: module 'explainer.collect' has no attribute ...` or `unexpected keyword argument`	A test file is new but `collect.py` or `prom.py` is the old version	Replace the source files, then run `grep -c workload_history src/explainer/collect.py` (expect 2 or more)

Symptom	Cause	Fix
`error: cannot reach Prometheus at http://127.0.0.1:9090`	Prometheus is not running or not forwarded	Part A step 4; run `make dev` in the mixin repo if needed
`make collect` shows tiny values or `hist(h)` near 0	Pods are new, or Prometheus lost history on restart	Wait longer; use a shorter `WINDOW` such as `1h`
A workload is missing from the table	Pod not yet scraped (a CronJob pod appears every 20 minutes)	Wait for the next run
`leaky-memory` shows a peak far below its limit	Restarted container's old series was lost (fixed by merging)	Update to the latest `collect.py`
Restarts show `1.01`	`increase()` extrapolates	Fixed: restarts are rounded to whole numbers
KRR: `No module named 'click'` and many `Ignoring ... markers` lines	Python is newer than 3.12	Create a Python 3.12 environment (Part E)
KRR shows `Not enough data` or `?`	Less than 100 points of history, or short-lived CronJob pods	Wait 3 to 4 hours; for CronJobs this is expected
`error: model '...' is not installed`	Ollama does not have that model	`ollama ls`, then use `MODEL=<an installed name>`
`cannot reach Ollama at http://127.0.0.1:11434`	Ollama is not running	`ollama serve`
An explanation is flagged `CHECK FAILED` after a retry	The model broke a rule twice	Read the message; it names the decision, figure or reason. Try again or use another model
A benchmark case fails on `leaves out: ...` for a correct-looking text	The coverage keywords are crude	Check the text; if it is correct, extend the topic keywords in `explain.py`

The habit to build: read the first error line, not the last. The first line usually names the real cause, and later lines are consequences.

16. Expected results for the five lab workloads

After about 4 hours of history with KRR included, the `lab` profile should give these verdicts. Exact decimals vary a little between runs, but the verdicts should match.

Workload	CPU verdict	Memory verdict	Why it is correct
steady-overprovisioned	decrease_ok	decrease_ok	Near-zero CPU and about 22 MiB used against a 500m and 512 MiB request, with no blocking signal
throttled-cpu	decrease_blocked	decrease_ok	Throttled in about 60% of periods, so measured usage understates demand (`C_THROTTLED`)
leaky-memory	decrease_ok	increase_supported	OOM kills, repeated restarts, and a peak near 124 MiB against a 128 MiB limit (`S_OOM` , `S_NEAR_LIMIT`)
bursty-rare	increase_supported	decrease_ok	p95 usage above the 50m request; `K_DISAGREES` flags that KRR and the 5-minute p95 differ
batch-periodic	increase_supported	decrease_ok	CPU burns about 0.6 cores against a 0.2 request; memory is tiny. History is spread over many short runs

How to use this table: if your verdict for a workload differs, do not edit the rules to match. Open that workload's evidence pack and its verdict, and find which number or rule explains the difference. Understanding the difference is the point of the exercise.

A correct run of the explainer produces, for `batch-periodic` , text of this kind: the CPU request of 0.200 cores is below the p95 usage of 0.613 cores, and the memory candidate is below the 32Mi minimum floor, so the minimum is used. The wording can differ between models and prompt versions, but the figures and the decision must not.

17. Limits and what not to claim

This is a learning and portfolio project built on a small local cluster. Do not present it as a validated production tool. Know these limits before you explain it to anyone.

- **Lab scale.** Five workloads and hours of history. Production sizing needs days or weeks, which is why the `dev` and `production` profiles ask for 7 and 28 days.
- **Thresholds are illustrative.** They come from examples in the LearnKube rightsizing book, not from a standard. Tune them to your own risk tolerance.
- **The checks verify decisions, figures and reason coverage, not truth.** A sentence can quote correct figures and still mislead.
- **No latency, error-rate or SLO data.** A decrease that looks safe on resource metrics can still hurt users. Use this tool as one input, never the only one.
- **Workload names are inferred from pod names**, not from owner references. Revisions are not tracked, so data from different releases is mixed.
- **One model benchmarked** on 17 cases, 12 of which are invented. Treat the result as a baseline.
- **VPA is not integrated.** Workloads managed by a Horizontal Pod Autoscaler are not handled specially.
- **The tool never changes anything.** It recommends and explains. A human applies changes.

Why this section exists: an honest account of limits is what makes a project credible. When you present it, state the limits before someone else finds them.

18. Exercises

Do these after the main run. Each one teaches a different part of the system. Write down what you predict before you run it, then compare.

- [ ] **Change a threshold.** In `src/explainer/verdict.py`, change the default `throttle_block` in the `Policy` class from 0.05 to 0.70. Run `make verdict`. Which verdict changes and why? Then put it back.
- [ ] **Compare profiles.** Run `make verdict PROFILE=dev` and `PROFILE=production`. Explain every verdict that differs from `lab`.
- [ ] **Read an evidence pack.** Open `evidence/explainer-lab/throttled-cpu__app.json`. Find the throttle ratio, the p95 CPU and the CPU limit. Explain in two sentences why this workload must not be cut.
- [ ] **Add a workload.** Add a sixth Deployment to `scenarios/explainer-lab.yaml` with a CPU limit of 500m and no heavy load. Predict its verdict, deploy it, wait an hour, and check.

- [ ] **Break the checker on purpose.** In `tests/test_faithful.py`, add a case where the model writes `0.409 millicores`. Confirm the checker rejects it, and explain why a unit mismatch matters.
- [ ] **Find a disagreement.** Compare KRR's number and this tool's own number for each workload. Which two differ most, and what could cause the gap?
- [ ] **Add a synthetic case.** In `src/explainer/synthetic.py`, add a case for a workload with a high CPU limit and a rising memory trend. Predict the verdict, add it to the expected list in `tests/test_faithful.py`, and run `make test`.
- [ ] **Explain it aloud.** In five minutes, tell a colleague why the model is not allowed to make the decision.

19. References

- Kubernetes rightsizing guide (LearnKube): the source of the request, limit, throttling and policy concepts used here.
- KRR, Kubernetes Resource Recommender (Robusta): the open-source tool whose output the project reads.
- Resource review mixin: the repository that provides the kind cluster with Prometheus and Grafana.
- Ollama: runs the local models.

Project files: `README.md` in the project root has the architecture diagram, the benchmark and the roadmap. The `tests/` folder is the best specification of how each part is supposed to behave.